

Tarea Corta 04: Implementación de semáforos

https://github.com/sergiopb06/Tareas_Paralela_2026.git

Sergio Peralta Barboza

Escuela de Ciencias de la Computación e Informática, Universidad de Costa Rica

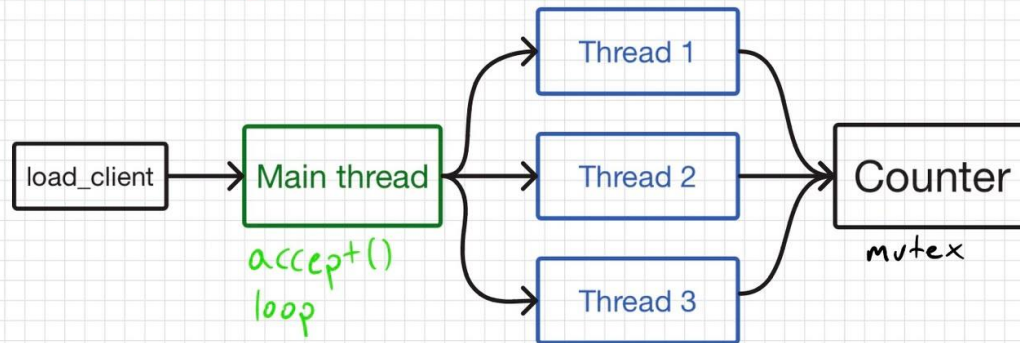
CI-0117 Programación Paralela y Concurrente, Grupo 01

Ing. Sleyter Angulo, M.Sc

Jueves 1 de octubre de 2026

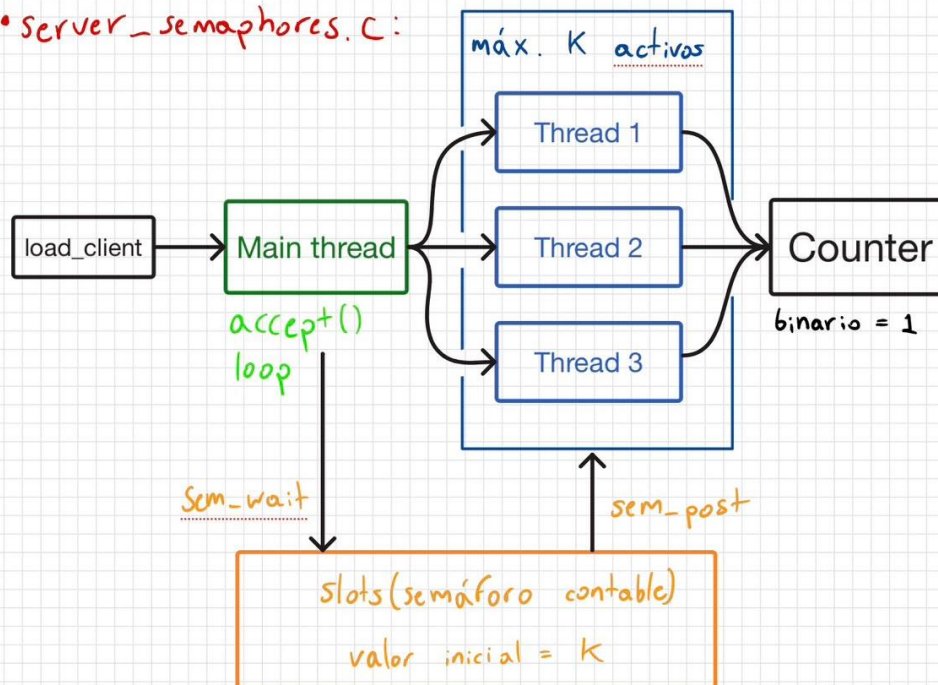
Diseño de la solución: server_semaphores vs. server_safe

• *Server-safe.c:*



- for each request, a thread is created: Thread-per-connection

• *server_semaphores.c:*



- Creates a max of K threads. If K threads are active, no more connections are active.
- Each thread handles a connection, once it's done it frees its space from slots.

Experimentos:

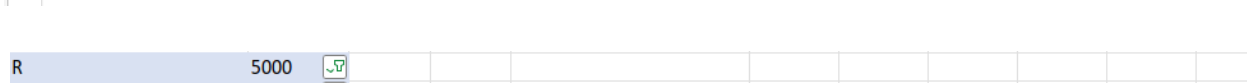
Se llevaron a cabo 36 pruebas en server_safe y 144 pruebas en server_semaphores; utilizando ambos resultados, se comparará el desempeño entre estos servidores. Para ello se hicieron las pruebas utilizando los siguientes parámetros:

- N = cantidad de cores.
- T = cantidad de hilos.
- R = cantidad de paquetes por hilo.
- K = cantidad de slots del counting_semaphore (únicamente para server_semaphores)

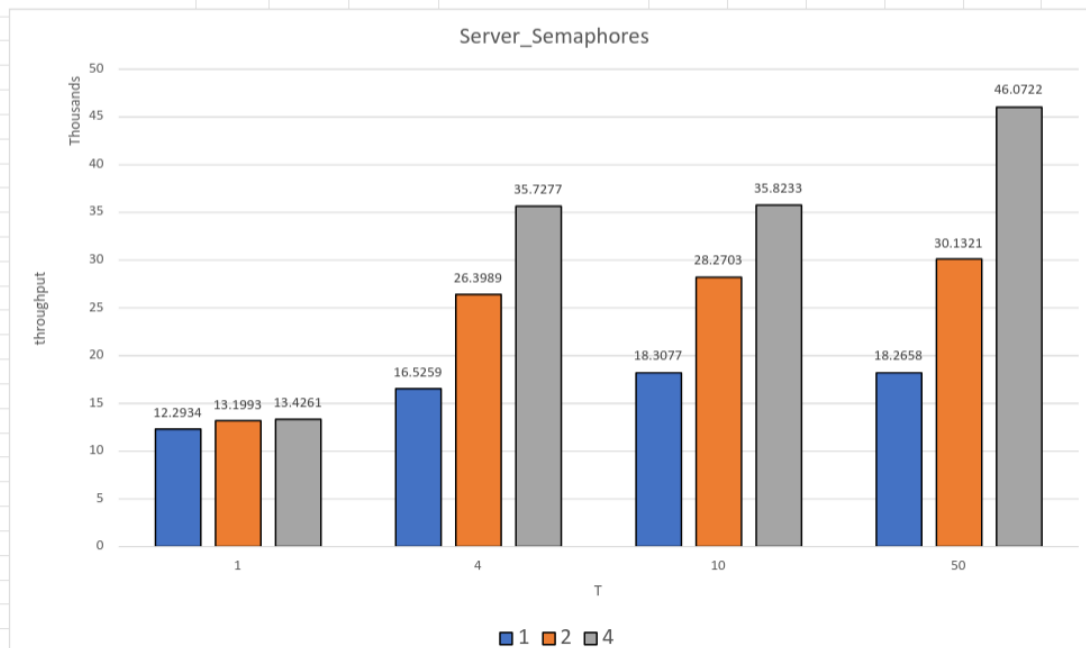
Los valores que estas variables tomaban son:

- N = {1, 2, 4}
- T = {1, 4, 10, 50}
- R = {100, 500, 5000}
- K = {1, 4, 16, 65}

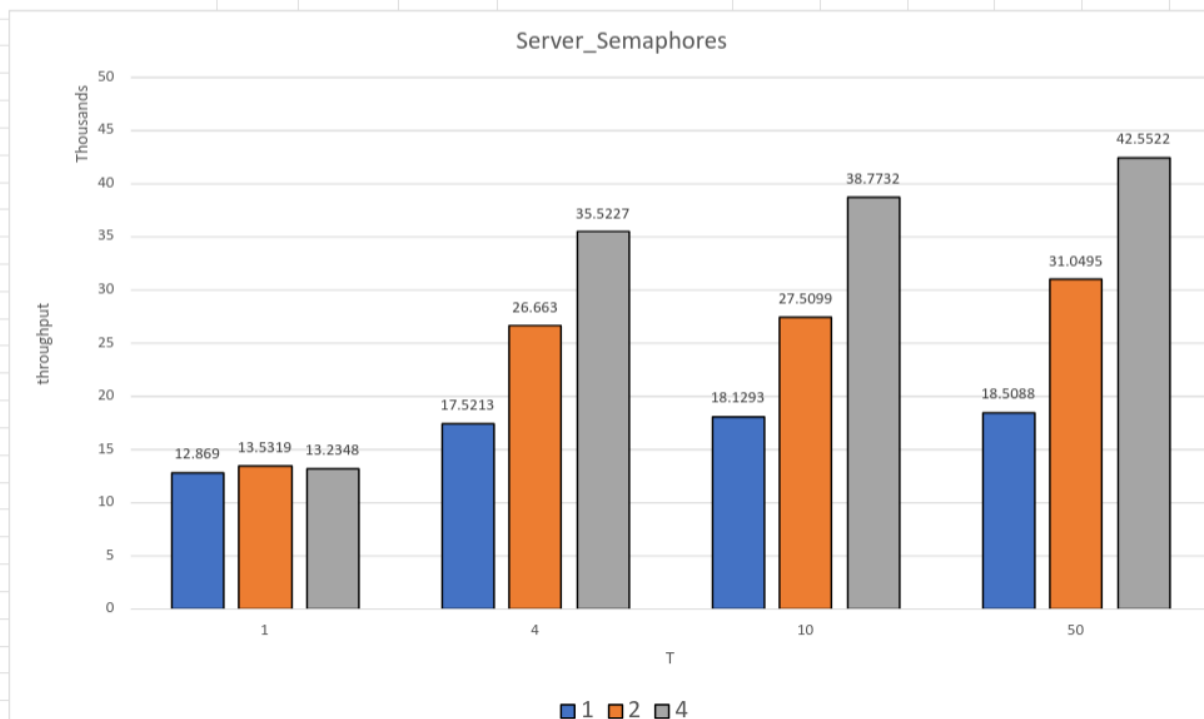
Para promediar el tiempo de cada experimento, se realizaron 3 ejecuciones del mismo caso y se promedió su tiempo. Los datos analizados en las gráficas y tablas son el resultado del throughput del experimento; este se consigue dividiendo la cantidad de requests ($T \times R$) entre el promedio del tiempo de ejecución. Algunos ejemplos realizados, mas no todos, son los siguientes:



Average of throughput	N			
T		1	2	4 Promedio de rendimiento
1	12293.4	13199.3	13426.1	12972.93333
4	16525.9	26398.9	35727.7	26217.5
10	18307.7	28270.3	35823.3	27467.1
50	18265.8	30132.1	46072.2	31490.03333
Promedio de rendimiento	16348.2	24500.15	32762.325	24536.89167

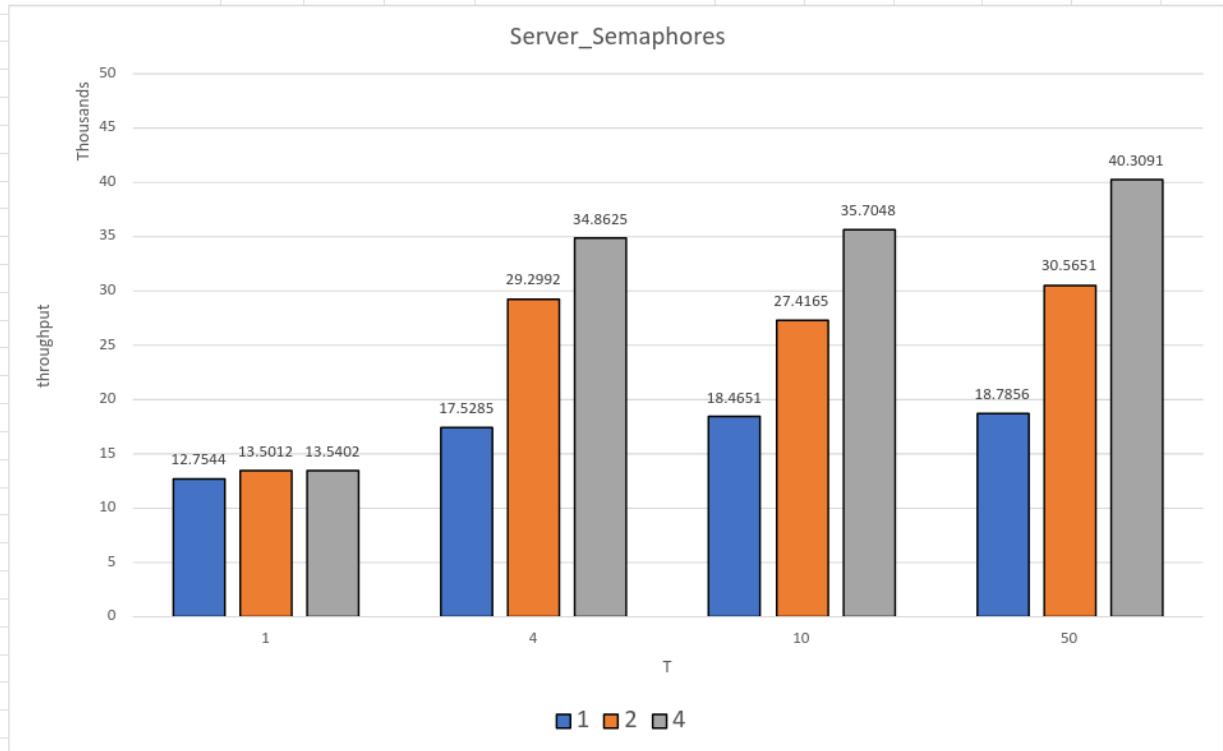


Average of throughput	N	1	2	4	Promedio de rendimiento
1	12869	13531.9	13234.8	13211.9	
4	17521.3	26663	35522.7	26569	
10	18129.3	27509.9	38773.2	28137.46667	
50	18508.8	31049.5	42552.2	30703.5	
Promedio de rendimiento	16757.1	24688.575	32520.725	24655.46667	

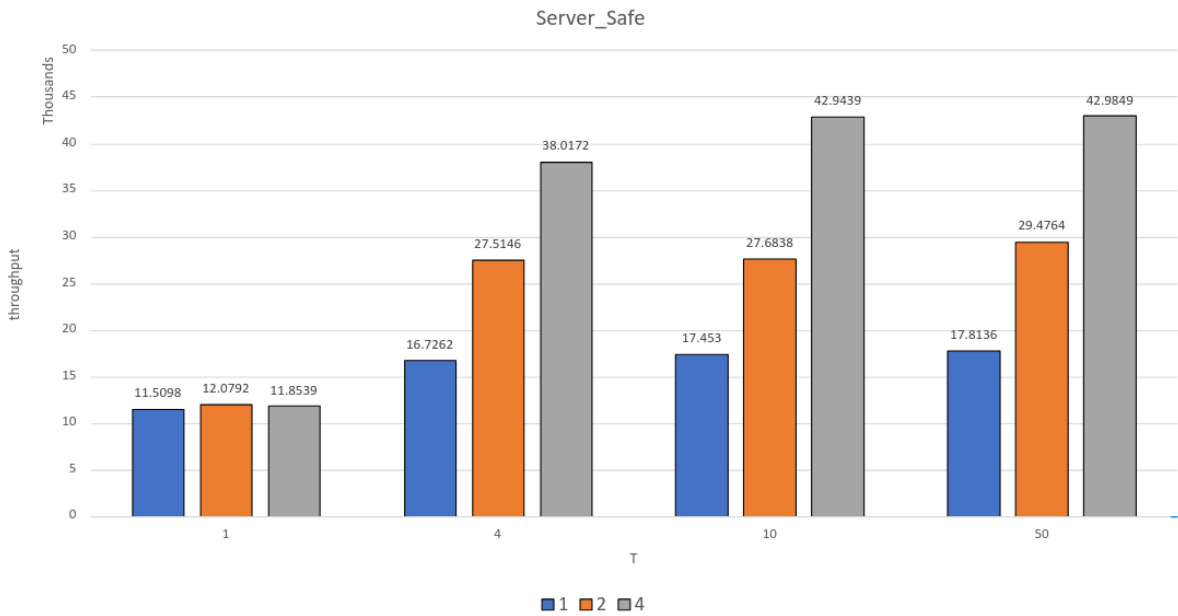


R	5000	5000
K	64	64

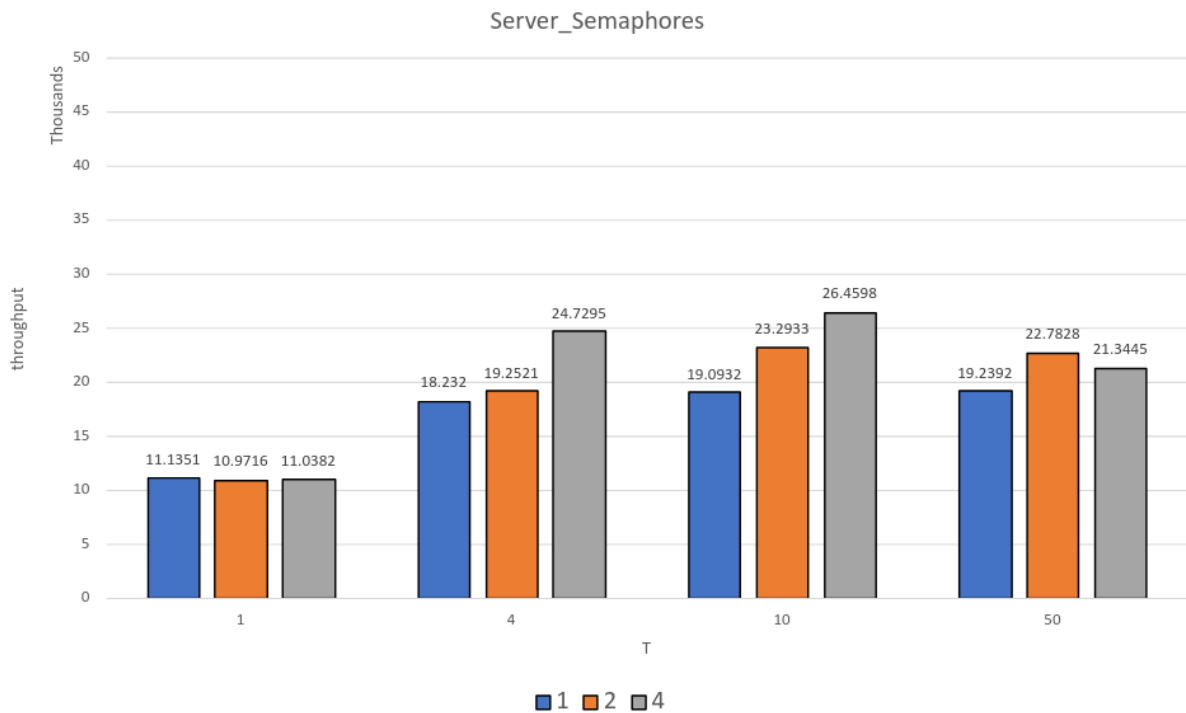
Average of throughput	N				
T		1	2	4	Promedio de rendimiento
	1	12754.4	13501.2	13540.2	13265.26667
	4	17528.5	29299.2	34862.5	27230.06667
	10	18465.1	27416.5	35704.8	27195.46667
	50	18785.6	30565.1	40309.1	29886.6
Promedio de rendimiento		16883.4	25195.5	31104.15	24394.35



R	500				
Average of throughput	N				
T		1	2	4	Promedio de rendimiento
1	11509.8	12079.2	11853.9		11814.3
4	16726.2	27514.6	38017.2		27419.33333
10	17453	27683.8	42943.9		29360.23333
50	17813.6	29476.4	42984.9		30091.63333
Promedio de rendimiento	15875.65	24188.5	33949.975		24671.375

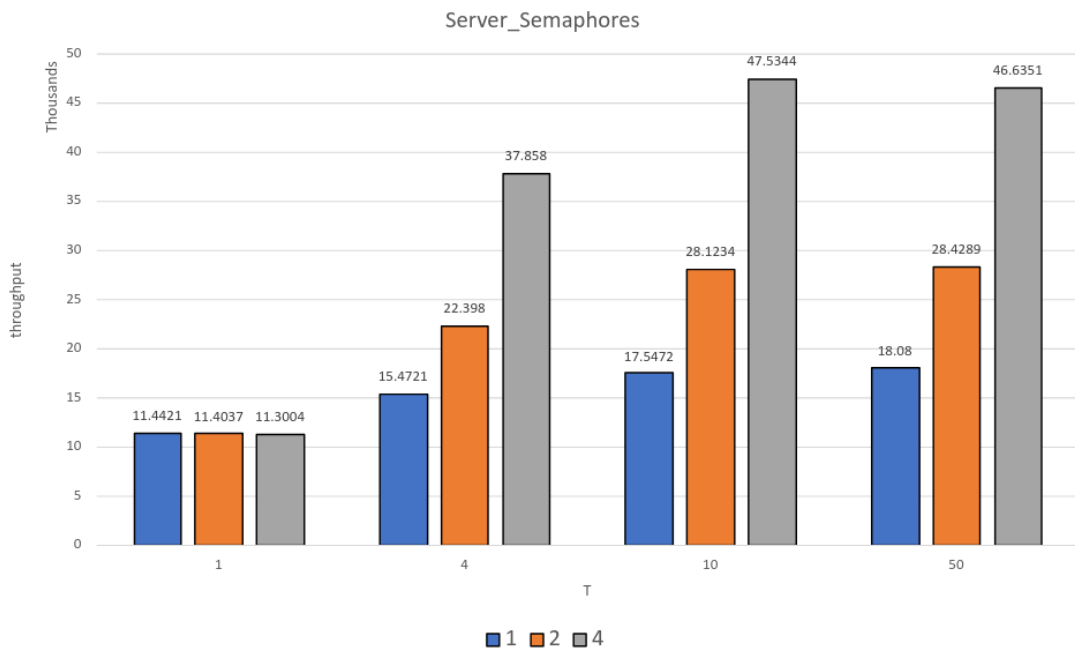


R	500				
K	1				
Average of throughput	N				
T		1	2	4	Promedio de rendimiento
1	11135.1	10971.6	11038.2		11048.3
4	18232	19252.1	24729.5		20737.86667
10	19093.2	23293.3	26459.8		22948.76667
50	19239.2	22782.8	21344.5		21122.16667
Promedio de rendimiento	16924.875	19074.95	20893		18964.275



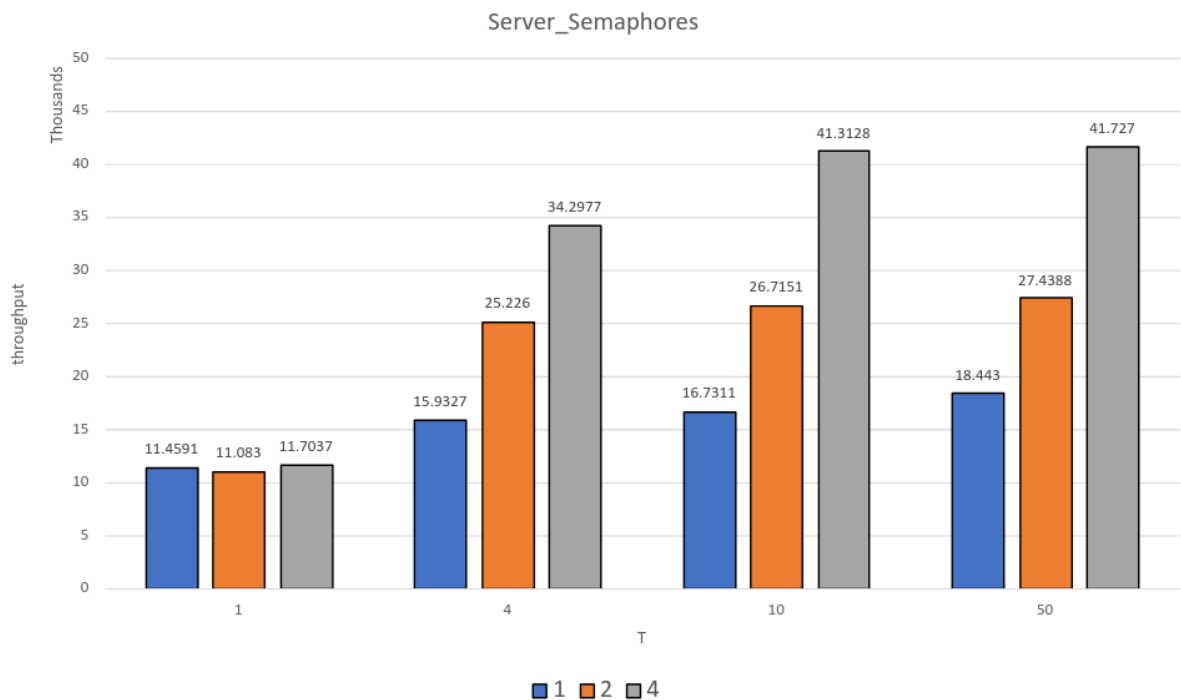
R	500	✓
K	4	✓

Average of throughput	N				
T		1	2	4	Promedio de rendimiento
	1	11442.1	11403.7	11300.4	11382.06667
	4	15472.1	22398	37858	25242.7
	10	17547.2	28123.4	47534.4	31068.33333
	50	18080	28428.9	46635.1	31048
Promedio de rendimiento		15635.35	22588.5	35831.975	24685.275



R	500	✓
K	16	✓

Average of throughput	N				
T		1	2	4	Promedio de rendimiento
	1	11459.1	11083	11703.7	11415.26667
	4	15932.7	25226	34297.7	25152.13333
	10	16731.1	26715.1	41312.8	28253
	50	18443	27438.8	41727	29202.93333
Promedio de rendimiento		15641.475	22615.725	32260.3	23505.83333



The chart displays throughput values for different values of T (1, 4, 10, 50) across three configurations (1, 2, 4). The y-axis represents throughput in thousands, ranging from 0 to 50. The x-axis represents T. The legend indicates that configuration 1 is blue, configuration 2 is orange, and configuration 4 is gray.

T	Configuration 1 (Blue)	Configuration 2 (Orange)	Configuration 4 (Gray)
1	10.4591	11.7478	11.5759
4	16.326	25.6496	32.5818
10	17.1674	25.6973	42.3589
50	17.9007	26.8856	38.7409

Para interpretar las tablas y los resultados, cada celda es un valor con T threads y N cores, utilizando los parámetros del número de requests por hilo y los slots disponibles para el counting_semaphore. Estos parámetros afectan directamente al rendimiento en cada celda. Cuanto más alto sea el valor del throughput, mejor, ya que significa que logra procesar más requests por segundo. Como se mencionó, los ejemplos mostrados no son todos los experimentos realizados; dentro del documento “results_safe.xlsx” y “results_semaphores.xlsx” se pueden ajustar manualmente los parámetros y comparar resultados. Las conclusiones realizadas se basan en la comparación de resultados utilizando todos los experimentos.

Conclusiones:

No se puede concluir nada con certeza sobre la influencia de K, ya que al hacer solo tres repeticiones de cada caso, los resultados pueden estar afectados por ruido; estas conclusiones son hipótesis sobre el comportamiento del servidor.

- Al utilizar $K = 1$, el servidor se vuelve secuencial. Esto se nota de forma clara cuando comparamos con server_safe: con 4 núcleos, la diferencia es de entre 13 000 y 22 000 de throughput (por ejemplo, con $R = 5\,000$ y $T = 50$, 25 074 contra 46 888), y el rendimiento casi no mejora al aumentar los núcleos. Es el resultado más claro, porque se repite en todas las cargas.
- Se observa una tendencia interesante con $K = 4$: con 4 núcleos y $T = 50$, el servidor iguala o supera a server_safe en las tres cargas de R (46 072 contra 46 888 con $R = 5\,000$; 46 635 contra 42 985 con $R = 500$; 47 900 contra 41 034 con $R = 100$) y rinde mejor que con $K = 16$ y $K = 64$. Una hipótesis es que al limitar la concurrencia al mismo número de núcleos se reduce la competencia por CPU y semáforos.
- Con $K = 16$, los resultados son similares a server_safe con 1 y 2 núcleos, pero con 4 núcleos presenta un peor rendimiento.
- Con $K = 64$ ocurre algo parecido: resultados similares a server_safe con 1 y 2 núcleos y peor rendimiento con 4. Como con $T \leq 50$ el límite nunca se alcanza,

esta diferencia no se debe a que K frene hilos, sino probablemente al costo de las operaciones extra sobre el counting_semaphore.

- Existen algunos comportamientos inesperados; por ejemplo, con $R = 5\,000$, $T = 10$, $N = 4$ y $K = 4$, el throughput es 35 823. Con los mismos parámetros ($T = 10$, $N = 4$ y $K = 4$) pero cambiando el número de paquetes de 5 000 a 500, se observa una mejora considerable: el throughput pasa a 47 534. Este comportamiento es inesperado, ya que con menos paquetes por hilo se esperaría un throughput igual o menor (con $T = 1$, por ejemplo, un R pequeño da menor throughput por costos de arranque del servidor). Esto sugiere que la diferencia se debe a ruido de medición y no a un efecto de R , y refuerza que los resultados no son concluyentes.

Limitaciones:

- Cada experimento se repitió solo 3 veces. Se puede encontrar ruido en los resultados.
- El cliente corre con dos núcleos físicos; esto podría limitar el rendimiento en casos con varios núcleos.